

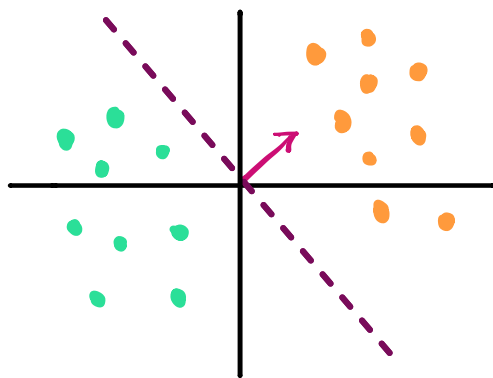
Perceptron

We made a generative assumption for Naive Bayes but ended up getting a linear classifier for $P(y|x)$. So let's try to model a linear classifier directly.

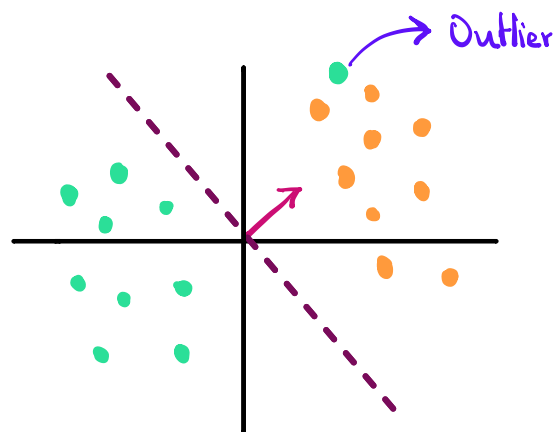
There is some unknown but fixed w that generates the labels given the datapoints.

$$\text{ie. } y = 1 \text{ if } w^T x \geq 0 \\ y = 0/1 \text{ otherwise}$$

Linear Separability Assumption



Allowed



Not allowed

$$\min_{h \in \mathcal{H}_{\text{linear}}} \sum_{i=1}^n \mathbb{1}(h(x_i) \neq y_i) \rightarrow \text{NP hard problem even when } h \text{ is linear for a general dataset}$$

But now we have a linear separability assumption.

This means there EXISTS a line that can linearly separate the data.

If we pick that $w^* : w^{*T} x_i - y_i = 0$. How can we find this w ?

↪ best w

Perceptron Algorithm

Input: $\{(x_1, y_1) \dots (x_n, y_n)\}$ $x_i \in \mathbb{R}^d$ and $y_i \in \{+1, -1\} \rightarrow$ preferred $\{+1, 0\}$

$w_0: \vec{0} \in \mathbb{R}^d$ ie $[0, 0 \dots 0]$

Repeat until convergence:

- Pick (x_i, y_i) pair from the dataset
- if $\text{sign}(w_t^T x_i) = y_i$: do Nothing
- else: $w_{t+1} = w_t + x_i y_i$ $\rightarrow$ update rule

Explanation:

- 1) if w_t classifies all the points correctly: algorithm ends
- 2) if it makes a mistake on any datapoint: take the datapoint, multiply it by y_i and add the result to w_t . This becomes w_{t+1} .

Eg: we have $(x_1, y_1) \dots (x_{10}, y_{10})$

let's say in the 4th iteration, w_4 correctly classifies $x_1 \dots x_6$

It makes a mistake on x_7 (i.e. $\text{sign}(w_4^T x_7) \neq y_7$)

Update: $w_5 = w_4 + y_7 \cdot x_7$

Proceed to check for x_8, x_9 and x_{10}

Start the process again from x_1 and update if there's a mistake.

The claim is that we can keep doing this until convergence

Update Rule

If mistake : $w_{t+1} = w_t + x_i \underline{y_i} \rightarrow \{+1, -1\}$

Mistake type 1:

Prediction = +1 $w_t^T \cdot x_i \geq 0$

Actual = -1 $y_i = -1$

$$w_{t+1} = w_t - x_i$$

Why w_{t+1} is better than w_t ?

$$w_t^T \cdot x_i \geq 0 \text{ but } w^{*T} \cdot x_i < 0$$

$$w_{t+1}^T \cdot x_i = (w_t - x_i)^T \cdot x_i$$

$$= w_t^T \cdot x_i - \|x_i\|^2$$

$$\therefore w_{t+1}^T \cdot x_i < w_t^T \cdot x_i$$

$w_{t+1}^T \cdot x_i$ may not be < 0 but it is moving in the right direction

Mistake type 2:

Prediction = -1 $w_t^T \cdot x_i < 0$

Actual = +1 $y_i = +1$

$$w_{t+1} = w_t + x_i$$

Why w_{t+1} is better than w_t ?

$$w_t^T \cdot x_i < 0 \text{ but } w^{*T} \cdot x_i \geq 0$$

$$w_{t+1}^T \cdot x_i = (w_t + x_i)^T \cdot x_i$$

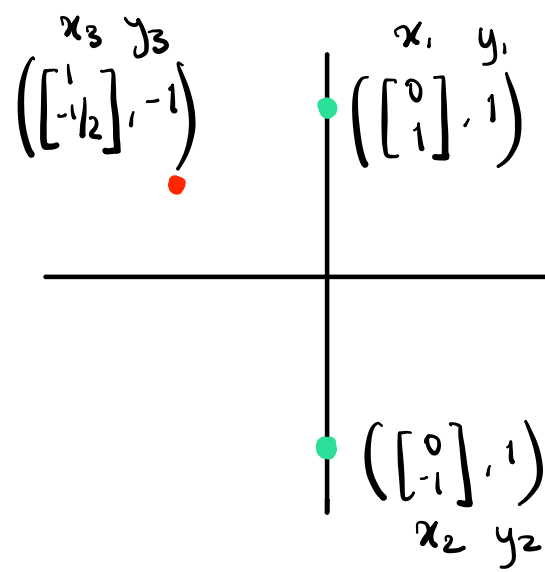
$$= w_t^T \cdot x_i + \|x_i\|^2$$

$$\therefore w_{t+1}^T \cdot x_i > w_t^T \cdot x_i$$

$w_{t+1}^T \cdot x_i$ may not be ≥ 0 but it is moving in the right direction

$\therefore$ Update rule pushes w in the right direction for x_i

Possible Flaw: w_{t+1} is a better w for the specific (x_i, y_i) that w_t made a mistake on. It may be worse for some of the earlier points that w_t correctly classified by w_t .



Conclusion: w never converges
Oscillates between $(-1, \frac{1}{2})$ & $(-1, -\frac{1}{2})$

$$w_0 = [0, 0]$$

$$w_0^T x_1 = (0, 0)^T (0, 1) = 0, \hat{y}_1 = 1, y_1 = 1$$

$$w_0^T x_2 = (0, 0)^T (0, -1) = 0, \hat{y}_2 = 1, y_2 = -1$$

$$w_0^T x_3 = (0, 0)^T (1, -\frac{1}{2}) = 0, \hat{y}_3 = 1, y_3 = -1$$

$$w_1 = w_0 + x_3 y_3 = [0, 0] - [1, -\frac{1}{2}] = [-1, \frac{1}{2}]$$

$$w_1^T x_1 = (-1, \frac{1}{2})^T (0, 1) = \frac{1}{2}, \hat{y}_1 = 1, y_1 = 1$$

$$w_1^T x_2 = (-1, \frac{1}{2})^T (0, -1) = -\frac{1}{2}, \hat{y}_2 = -1, y_2 = -1$$

$$w_2 = w_1 + x_2 y_2 = [-1, \frac{1}{2}] + [0, -1] = [-1, -\frac{1}{2}]$$

$$w_2^T x_3 = [-1, -\frac{1}{2}] (1, -\frac{1}{2}) = -\frac{1}{2}, \hat{y}_3 = -1, y_3 = -1$$

$$w_2^T x_1 = [-1, -\frac{1}{2}] (0, 1) = -\frac{1}{2}, \hat{y}_1 = -1, y_1 = 1$$

$$w_3 = w_2 + x_1 y_1 = [-1, -\frac{1}{2}] + [0, 1] = [-1, \frac{1}{2}]$$

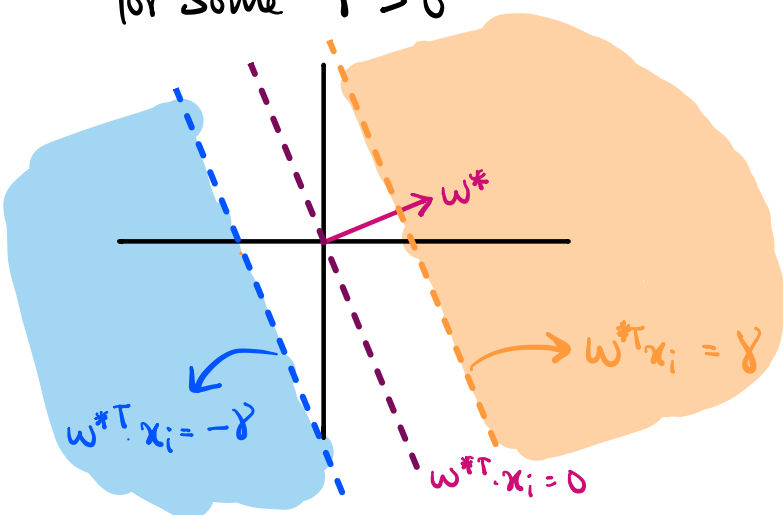
Not a "strictly linearly separable" dataset i.e. it is only linearly separable due to the tie-breaker condition.

Loose definition (only for understanding)

There should be no tie-breaker condition. Either $w_*^T x_i > 0$ or $w_*^T x_i < 0 \quad \forall i$.

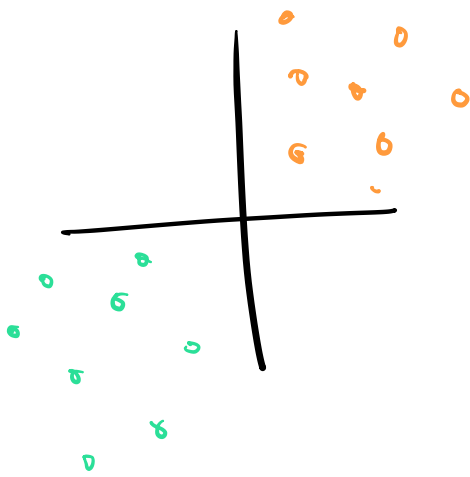
Formal definition (for Linear Separability)

There exists a $w^* \in \mathbb{R}^d$ such that $(w^{*T} \cdot x_i) \cdot y_i \geq \gamma \quad \forall i$
for some $\gamma > 0$



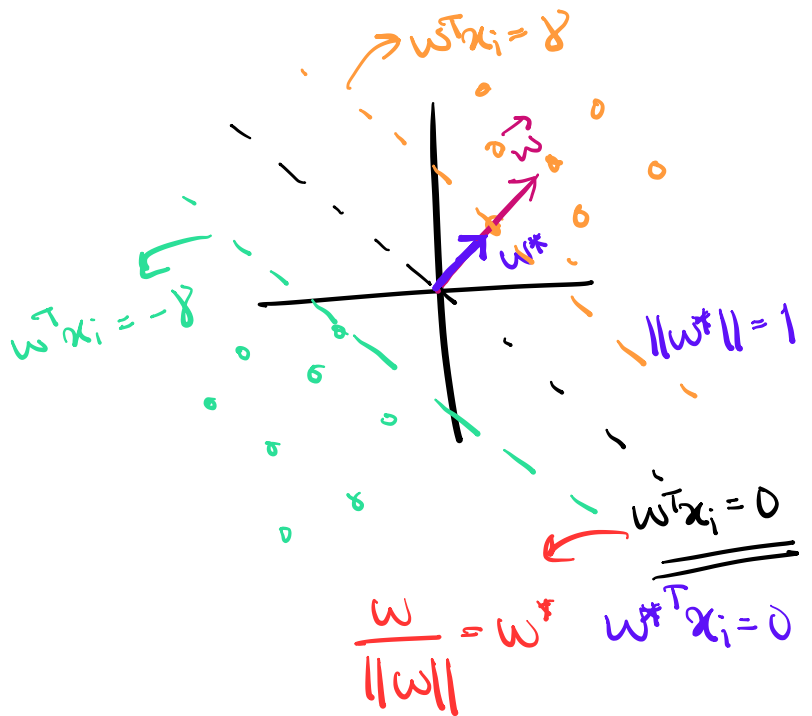
This is called Strict Linear Separability Assumption and it is sufficient to prove that perceptron converges.

i. Radius :

$$\forall i \in \text{Dataset}$$
$$\|x_i\| \leq R \text{ for some } R > 0$$


ii. $\|w^*\| = 1$

$w \Rightarrow$ that linearly separates the dataset. Then I can always find another w (w^*) in the same direction, but with the constraint that $\|w^*\| = 1$



Radius-Margin Bound (Rosenblatt, 1950s)

- * Update happens only when a mistake occurs
- * w_ℓ is the current guess and a mistake happens wrt (x, y)

$$\therefore w_{\ell+1} = w_\ell + x \cdot y \rightarrow \text{update rule}$$

$$\|w_{\ell+1}\|^2 = \|w_\ell + x \cdot y\|^2 = (w_\ell + x \cdot y)^T (w_\ell + x \cdot y)$$

$$= \|w_\ell\|^2 + 2 \cdot \underbrace{(w_\ell^T \cdot x) \cdot y}_{\text{Always} = +1} + \|x\|^2 \cdot \underbrace{y^2}_{\text{Always} = +1}$$

Either $w_\ell^T \cdot x > 0$ but $y = -1$

or $w_\ell^T \cdot x < 0$ but $y = +1$

Either way, this term < 0

$$\|w_{\ell+1}\|^2 \leq \|w_\ell\|^2 + \|x\|^2 \Rightarrow \|w_{\ell+1}\|^2 \leq \|w_\ell\|^2 + R^2$$

Sq. L_2 Norm of w in the $\ell+1^{\text{th}}$ iteration CANNOT be greater than
Sq. L_2 Norm of w in the ℓ^{th} iteration plus Radius².

By extension: $\|w_\ell\| \leq \|w_{\ell-1}\| + R$

$$\|w_1\|^2 \leq \|w_0\|^2 + R^2, \quad \|w_2\|^2 \leq \|w_1\|^2 + R^2 \text{ and so on}$$

$$\|w_2\|^2 \leq \|w_0\|^2 + 2 \cdot R^2$$

$$\therefore \|w_{\ell+1}\|^2 \leq (\ell+1) \cdot R^2 \quad \text{or} \quad \boxed{\|w_\ell\|^2 \leq \ell \cdot R^2} \rightarrow \textcircled{1}$$

#mistakes or # iterations

$$\begin{aligned}
 w_{l+1}^T \cdot w^* &= (w_l + \alpha \cdot y)^T \cdot w^* \\
 &= w_l^T \cdot w^* + \underbrace{(w^{*T} \cdot x) \cdot y}_{\geq \gamma} \rightarrow \text{By definition}
 \end{aligned}$$

$$w_{l+1}^T \cdot w^* \geq w_l^T \cdot w^* + \gamma$$

Again, by extension : $w_l^T \cdot w^* \geq w_{l-1}^T \cdot w^* + \gamma$

$$w_1^T \cdot w^* \geq w_0^T \cdot w^* + \gamma, \quad w_2^T \cdot w^* \geq w_1^T \cdot w^* + \gamma$$

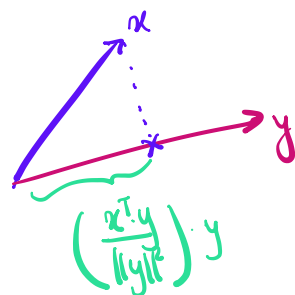
$$\therefore w_2^T \cdot w^* \geq w_0^T \cdot w^* + 2\gamma$$

$$\therefore w_{l+1}^T \cdot w^* \geq \underbrace{w_0^T \cdot w^*}_{\substack{\downarrow \\ \text{vector of all zeroes}}} + (l+1)\gamma$$

$$w_{l+1}^T \cdot w^* \geq (l+1) \cdot \gamma \quad \text{OR} \quad \boxed{w_l^T \cdot w^* \geq l \cdot \gamma} \rightarrow \textcircled{2} \text{ Margin Bound}$$

For any $(x, y) : \left\| \left(\frac{x^T y}{\|y\|^2} \right) \cdot y \right\|^2 \leq \|x\|^2$

$$\therefore (x^T y)^2 \leq \|x\|^2 \cdot \|y\|^2 \rightarrow \text{Cauchy-Schwartz}$$



$$\therefore w_l^T \cdot w^* \leq \|w_l\|^2 \cdot \|w^*\|^2 \Rightarrow w_l^T w^* \leq \|w_l\|^2 \times 1$$

$$\therefore \text{if } l \cdot \gamma \leq w_l^T w^* \Rightarrow l \gamma \leq \|w_l\| \Rightarrow l^2 \gamma^2 \leq \|w_l\|^2$$

$$\therefore l^2 \gamma^2 \leq \|w_l\|^2 \leq l R^2$$

$$\therefore l^2 \gamma^2 \leq l \cdot R^2 \quad \underline{\text{ie}} \quad \boxed{l \leq \frac{R^2}{\gamma^2}} \quad \text{Radius-Margin Bound}$$

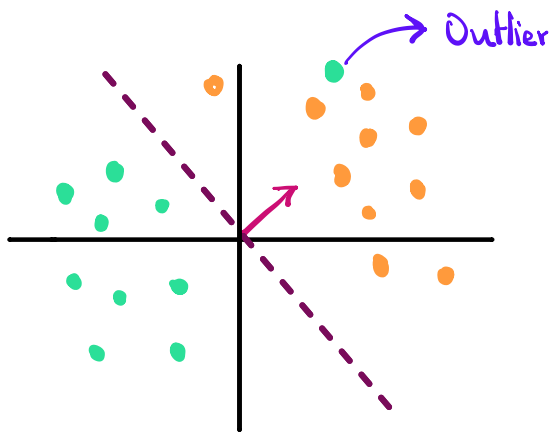
l is both # iterations and # mistakes. $\therefore$ Perceptron converges

Logistic Regression

So far we have used a strict linear separability assumption

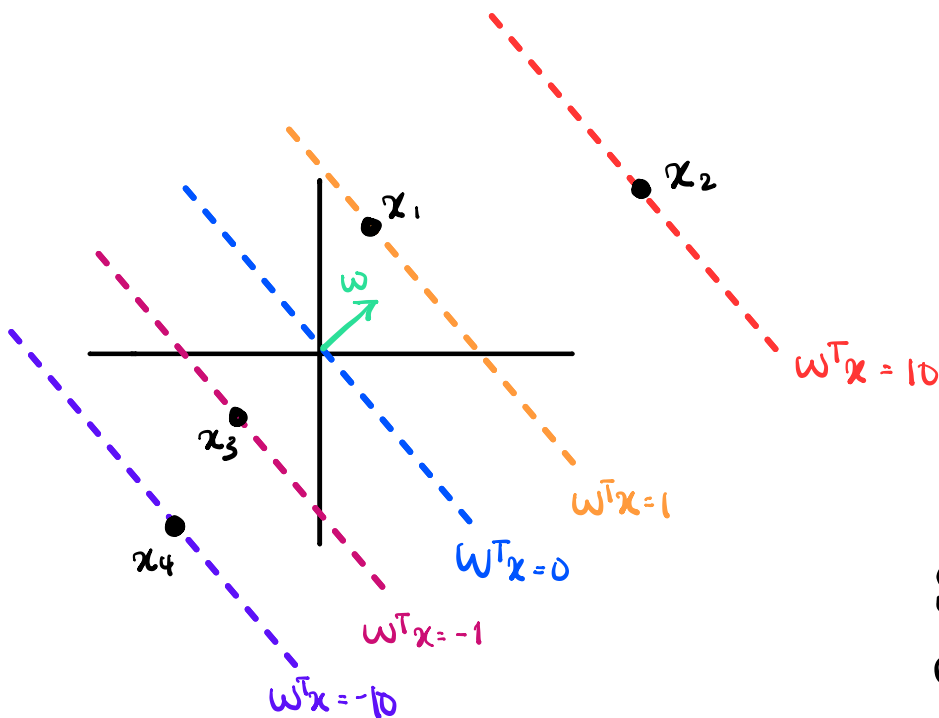
$$\begin{array}{l|l} y = 1 & \text{if } w^T x \geq 0 \\ y = 0 & \text{otherwise} \end{array} \quad \left| \quad \begin{array}{l} P(y=1) = 1, \text{ if } w^T x \geq 0 \\ P(y=1) = 0, \text{ otherwise} \end{array} \right.$$

Due to this strict linear separability assumption, we cannot model datasets like this:



Can we somehow relax this assumption?

Can we model the probabilities differently?



x_2 is more likely to belong to +1 than x_1 (ie) it is less likely to be an outlier.

Because $w^T x_2 > w^T x_1$

Similarly x_4 has a higher chance of belonging to -1 than x_3 . $w^T x_4 < w^T x_3$

Every point has some probability of being assigned +1 or 0
The probability will be decided by how high/low the score for each point is.

$$P(y_1=1) > P(y_2=1) > P(y_3=1) > P(y_4=1)$$

$$P(y_1=0) < P(y_2=0) < P(y_3=0) < P(y_4=0)$$

Moving in the same direction as $\vec{w}$ should increase $P(y_i=1)$
and moving in the opposite direction will increase $P(y_i=0)$

Function to convert score ($z = w^T x_i$) to $P(x_i \in 1)$

$z \in (-\infty, \infty)$. We need a function $g(z)$ to map this to $[0, 1]$

↪ link function

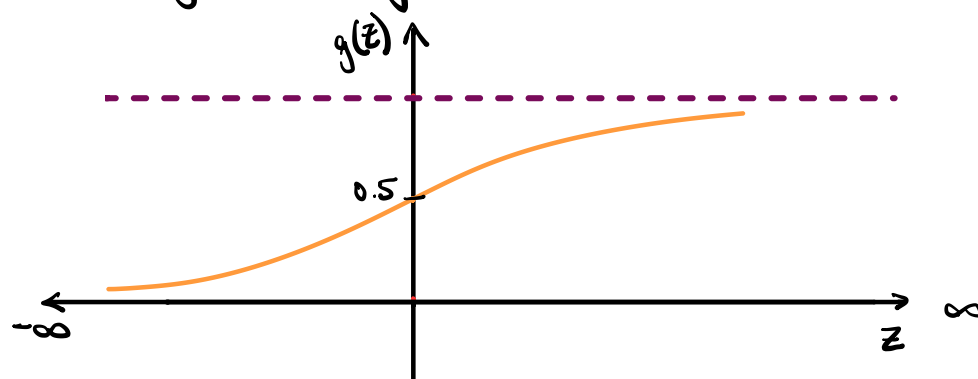
Desirable properties for the link function

1. $g(z) = 0.5$ if $z = 0$
2. $g(z) \rightarrow 1$ as $z \rightarrow \infty$
3. $g(z) \rightarrow 0$ as $z \rightarrow -\infty$

$$z = w^T x$$

Several popular link functions exist

Sigmoid / Logistic function



$$g(z) = \frac{1}{1+e^{-z}} \quad \text{ie.}$$

$$g(w^T x) = \frac{1}{1+e^{-w^T x}}$$

If this is our generative model then every point in the dataset is generated like this.

to be precise - given the features, the labels are generated like this.

- 1) There is an unknown but fixed w .
- 2) We are given the $\vec{x}_i \in \mathbb{R}^d$.
- 3) This point x_i gets a score $w^T x_i \in \mathbb{R}$
- 4) This score is converted into a probability

$$P(y_i = 1 | x_i) = \frac{1}{1 + e^{-w^T x_i}}$$

Naturally, $P(y_i = 0 | x_i)$ is just $1 - P(y_i = 1 | x_i)$

Again, for different w 's, the chances of seeing this exact dataset would be different. No prizes for guessing, this smells like a maximum likelihood problem - and it is!

Logistic Regression

$$P(y_i = 1 | x_i) = \frac{1}{1 + e^{-w^T x_i}}$$

$$\therefore P(\text{dataset} | w) = \prod_{i=1}^n P(y_i = 1 | x_i)^{y_i} \times P(y_i = 0 | x_i)^{1-y_i}$$

$$L(w) = \prod_{i=1}^n \left(\frac{1}{1 + e^{-w^T x_i}} \right)^{y_i} \cdot \left(\frac{e^{-w^T x_i}}{1 + e^{-w^T x_i}} \right)^{1-y_i}$$

$$\begin{aligned}
\log L(w) &= \sum_{i=1}^n y_i \cdot \log\left(\frac{1}{1+e^{-w^T x_i}}\right) + (1-y_i) \cdot \log\left(\frac{e^{-w^T x_i}}{1+e^{-w^T x_i}}\right) \\
&= \sum_{i=1}^n y_i \cdot (\log 1 - \log(1+e^{-w^T x_i})) + (1-y_i) (\log(e^{-w^T x_i}) - \log(1+e^{-w^T x_i})) \\
&= \sum_{i=1}^n y_i (-\log(1+e^{-w^T x_i})) + (1-y_i) (-w^T x_i - \log(1+e^{-w^T x_i})) \\
&= \sum_{i=1}^n y_i (-\log(1+e^{-w^T x_i})) + (-w^T x_i - \log(1+e^{-w^T x_i})) \\
&\quad - y_i (-w^T x_i - \log(1+e^{-w^T x_i})) \\
&= \sum_{i=1}^n \underbrace{y_i (-\log(1+e^{-w^T x_i}))}_{\text{orange}} + \underbrace{(-w^T x_i - \log(1+e^{-w^T x_i}))}_{\text{green}} \\
&\quad - \underbrace{y_i (-w^T x_i - \log(1+e^{-w^T x_i}))}_{\text{pink}} \quad \text{Cancel out} \\
&= \sum_{i=1}^n \underbrace{-w^T x_i}_{\text{blue}} + \underbrace{y_i (w^T x_i)}_{\text{pink}} - \underbrace{\log(1+e^{-w^T x_i})}_{\text{green}}
\end{aligned}$$

$$\log L(w) = \sum_{i=1}^n [(1-y_i)(-w^T x_i) - \log(1+e^{-w^T x_i})]$$

→ max this function over all w .

$\frac{\partial \log L(w)}{\partial w}$ does not have a closed form solution

Gradient based techniques can be used

Gradient Ascent update rule

$$\nabla \log L(w) = \sum_{i=1}^n \left[(1-y_i)(-x_i) - \left(\frac{e^{-w^T x_i}}{1 + e^{-w^T x_i}} \right) (-x_i) \right]$$

$$= \sum_{i=1}^n \left[-x_i + y_i x_i + x_i \left(\frac{e^{-w^T x_i}}{1 + e^{-w^T x_i}} \right) \right]$$

$$= \sum_{i=1}^n y_i x_i - x_i \left(\frac{1}{1 + e^{-w^T x_i}} \right)$$

$$\nabla \log L(w) = \sum_{i=1}^n x_i \left(y_i - \frac{1}{1 + e^{-w^T x_i}} \right)$$

$$w_{t+1} = w_t + \underbrace{\eta}_\text{step-size} \cdot \nabla \log L(w)$$

$$w_{t+1} = w_t + \eta \left(\sum_{i=1}^n x_i \left(\underbrace{y_i}_{\{0,1\}} - \underbrace{\frac{1}{1 + e^{-w_t^T x_i}}}_{g(w_t^T x_i)} \right) \right)$$

Observations:

1. Gradient is a linear combination of datapoints
2. How much each datapoint contributes to the gradient (i.e) the weight depends on how well our guess using the current weight w_t represents the true label y_i . Eg: if $y_i = +1$ and $g(w_t^T x_i) = 0.99$, then the weight is 0.01 and this point does not affect the gradient much.
More weight is given to the points where w makes (large) mistakes.

We get the solution $\hat{w}$ which can then be used to make predictions on the test points.

$$\text{sign}(w^T x_t) = \begin{cases} 1, & \text{if } w^T x_t \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Given x_{test} : $\hat{y}_{\text{test}} = \text{sign}(\hat{w}^T \cdot x_{\text{test}})$

Advantage of this model : we can also predict probabilities

(ie) confidence of the prediction

$$g(\hat{w}^T \cdot x_{\text{test}}) \rightarrow P(\text{class} = +1)$$
$$1 - g(\hat{w}^T \cdot x_{\text{test}}) \rightarrow P(\text{class} = 0)$$

$$w^* = \sum_{i=1}^n \alpha_i x_i \quad [\text{Representer Theorem}]$$

$\therefore$ Logistic Regression can be Kernelized

Regularized (or Bayesian) Logistic Regression

$$\min_w \sum_{i=1}^n [\log(1 + e^{-w^T x_i}) + w^T x_i (1 - y_i)] + \underbrace{\frac{\lambda}{2} \|w\|^2}_{L_2 \text{ penalty}}$$

cross-validated hyperparameter

Advantage over Perceptron

Works for any dataset (even not linearly separable) and works well in practice.

Binary Classification

(Refer "Why logreg does not solve binary classification" video)

→ min # mistakes

$$\min_w \sum_{i=1}^n \mathbb{1}(\underline{w^T x_i \neq y_i})$$

$$h(x_i) \neq y_i$$

$$h \sim \mathcal{H}_{\text{linear}}$$

$$h(x_i) = w^T x_i$$

1. KNN ✓

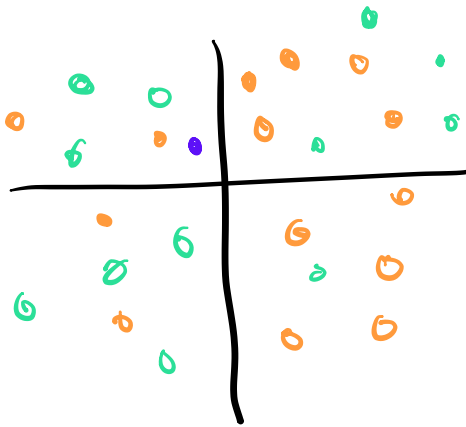
2. Decision Trees ✓

3. Naive Bayes / GNB

→ conditionally independent

4. Perceptron → linear separability

5. Logistic Regression → solves NP hard?



$k=3$

prediction = +1

Greedy Algorithm

⇒ Best outcome for the immediate decision

$$f_k \leq \theta$$

doesn't have the best info gain

$$f_l \leq \theta_2$$

$$f_m \leq \theta_3$$



min : NP hard
logreg